ClinicFlow — Project Overview

- Clinic appointment management SaaS for DevOps Capstone
- Book visits, track check-in, operate full delivery pipeline
- Student: Kushal Sacharya (kushals256)
- Goal: laptop → CI/CD → GHCR → AWS EKS → observe

Tech Stack

- Frontend: React + Vite + Nginx (non-root)
- Backend: FastAPI + SQLAlchemy + Alembic + Pytest
- Data: PostgreSQL 16
- DevOps: Docker, GitHub Actions, Trivy, GHCR
- Cloud: Terraform, AWS VPC + EKS, Helm, Ingress, HPA
- GitOps & Observability: Argo CD, Prometheus, Grafana

Architecture

- Developer → GitHub → Actions (test/build/scan/push)
- Terraform provisions VPC + EKS in ap-south-1
- Helm/Argo CD deploys frontend + backend + Postgres
- Ingress routes / to UI and /api to FastAPI
- Prometheus scrapes /metrics → Grafana dashboards

CI/CD Flow

- Trigger: push to main
- Job 1: pytest + frontend build (quality gate)
- Job 2: Docker build backend & frontend
- Trivy: fail on HIGH/CRITICAL (ignore-unfixed)
- Push SHA-tagged images to ghcr.io/kushals256/*

Infrastructure (Terraform)

- VPC with 2 public + 2 private subnets
- NAT gateway for private workload egress
- EKS 1.31 with managed node group (t3.medium)
- IaC only — no console click-ops for create
- terraform destroy after evaluation (cost control)

Kubernetes / Helm Setup

- Namespace: clinicflow
- Helm chart: helm/clinicflow (Deployments, Services, Ingress, HPA)
- ≥2 replicas for frontend and backend
- ClusterIP Services + nginx Ingress
- In-cluster Postgres PVC for classroom demo

Observability

- FastAPI /metrics via prometheus-fastapi-instrumentator
- ServiceMonitor selects clinicflow-backend
- kube-prometheus-stack + Grafana
- Dashboard: HTTP request rate + p95 latency
- Answers: traffic, latency, errors, HPA pressure

Key Learnings

- Own application domain matters (avoid TaskBoard clones)
- Tests before image push prevent broken promotions
- SHA tags give production ↔ git traceability
- Ingress is config; controller implements it
- Always destroy cloud resources after the demo